

CARMESIM QUEST AVANÇOS 2

MEMBROS DO GRUPO:

***Davi**

***Breno**

Objetivo desses avanços

- Facilitar nos avanços futuros do jogo criando funções, tipos e estruturas.
- Limpar o código e deixar ele mais legível
- Adição de novas regiões do jogo

Resumo do que mudou:

******Lista encadeada de textos

******Funções para definir a visibilidade de textos e imagens

******Funções para adicionar botões

******Funções para mudar os estados do jogo

Definindo os tipos para a lista encadeada

```
typedef struct TextNode {  
    TTR textTexture;  
    char nome[50];  
    bool visible;  
    int x, y;  
    struct TextNode* prox;  
} TextNode;  
  
typedef struct TextList {  
    TextNode* head;  
    int count;  
} TextList;
```

Iniciando a lista encadeada

```
TextList* createTextList() {  
    TextList* list = (TextList*) malloc(sizeof(TextList));  
    list->head = NULL;  
    list->count = 0;  
    return list;  
}
```

Função para renderizar texto por nome

```
void renderizarTextoPorNome(TextList* list, SDL_Renderer* renderer, const char* nome){  
    TextNode* textNode = buscaTexto(list, nome);  
    if(textNode && textNode->visible){  
        renderTextAt(renderer, textNode->textTexture, textNode->x, textNode->y);  
    }  
}
```

Funções para definir a visibilidade
dos textos e da imagens

Definir visibilidade das imagens

```
void definirVisibilidadeImagem(ListaImagens* lista, const char* nome, bool visivel){  
  
    Noimagem* atual = lista -> head;  
  
    while(atual != NULL){  
        if(strcmp(atual -> nome, nome) == 0){  
            atual -> imagem.visivel = visivel;  
            return;  
        }  
        atual = atual -> prox;  
    }  
    printf("Imagem '%s' não encontrada para definir visibilidade.\n", nome);  
}
```


Definir visibilidade das imagens

```
void renderizarImagensVisiveis(ListaImagens* lista, SDL_Renderer* renderer){

    Noimagem* atual = lista -> head;

    while(atual != NULL){
        if(atual -> imagem.visivel && atual -> imagem.textura){
            SDL_RenderCopy(renderer, atual -> imagem.textura, NULL, &atual -> imagem.posicao);
        }
        atual = atual -> prox;
    }
}
```

Definir visibilidade das imagens

```
void esconderTodasImagens(ListaImagens* lista) {  
  
    Noimagem* atual = lista -> head;  
    while(atual != NULL) {  
        atual -> imagem.visivel = false;  
        atual = atual -> prox;  
    }  
}
```

Definir a visibilidade dos textos

```
void RenderizarTodosOsTextosVisiveis(TextList* list, SDL_Renderer* renderer){
    TextNode* atual = list->head;
    while(atual != NULL){
        if(atual->visible){
            renderTextAt(renderer, atual->textTexture, atual->x, atual->y);
        }
        atual = atual->prox;
    }
}
```

Definir a visibilidade dos textos

```
void setTextVisibilidade(TextList* list, const char* nome, bool visible){  
    TextNode* textNode = buscaTexto(list, nome);  
    if(textNode){  
        textNode->visible = visible;  
    }  
}
```

Definir a visibilidade dos textos

```
void esconderTodosOsTextos(TextList* list) {  
    TextNode* atual = list->head;  
    while(atual != NULL) {  
        atual->visible = false;  
        atual = atual->prox;  
    }  
}
```

Funções de botões

definido o tipo de Botão

```
typedef struct Botao {  
    SDL_Rect rect;  
    const char* texto;  
    SDL_Color corFundo;  
    SDL_Color corBorda;  
    SDL_Color corTexto;  
    bool visivel;  
    bool ativo;  
} Botao;
```

Função para criar botões

```
Botao criarBotao(int x, int y, int largura, int altura, const char* texto, SDL_Color fundo, SDL_Color borda, SDL_Color textoCor) {  
    Botao btn;  
    btn.rect = (SDL_Rect){x, y, largura, altura};  
    btn.texto = texto;  
    btn.corFundo = fundo;  
    btn.corBorda = borda;  
    btn.corTexto = textoCor;  
    btn.visivel = true;  
    btn.ativo = true;  
    return btn;  
}
```


Função para auxiliar no clique do botão

```
bool cliqueBotao(Botao btn, int mouseX, int mouseY) {  
    if (!btn.visivel || !btn.ativo) return false;  
    return (mouseX >= btn.rect.x && mouseX <= btn.rect.x + btn.rect.w &&  
            mouseY >= btn.rect.y && mouseY <= btn.rect.y + btn.rect.h);  
}
```

Função para renderizar os botões

```
void renderizarBotao(SDL_Renderer* ren, Botao btn, TTF_Font* fonte) {  
    if (!btn.visivel) return;  
  
    SDL_SetRenderDrawColor(ren, btn.corFundo.r, btn.corFundo.g, btn.corFundo.b, 255);  
    SDL_RenderFillRect(ren, &btn.rect);  
  
    SDL_SetRenderDrawColor(ren, btn.corBorda.r, btn.corBorda.g, btn.corBorda.b, 255);  
    SDL_RenderDrawRect(ren, &btn.rect);  
  
    if (btn.texto && fonte) {  
        TTR textoRender = CarregaTexto(ren, fonte, btn.texto, btn.corTexto);  
        if (textoRender.texture) {  
            int textoX = btn.rect.x + (btn.rect.w - textoRender.largura) / 2;  
            int textoY = btn.rect.y + (btn.rect.h - textoRender.altura) / 2;  
            renderTextAt(ren, textoRender, textoX, textoY);  
            freeTextTexture(textoRender);  
        }  
    }  
}
```

Funções para mudar o estado do
jogo

Exemplo de função para carregar a vila

```
void carregarVila() {  
    esconderTodasImagens(imagens_globais);  
    esconderTodosOsTextos(textos_globais);  
    definirVisibilidadeImagem(imagens_globais, "Vila", true);  
    setTextVisibilidade(textos_globais, "vila_loja", true);  
    setTextVisibilidade(textos_globais, "vila_arena", true);  
    setTextVisibilidade(textos_globais, "vila_mapa", true);  
}
```

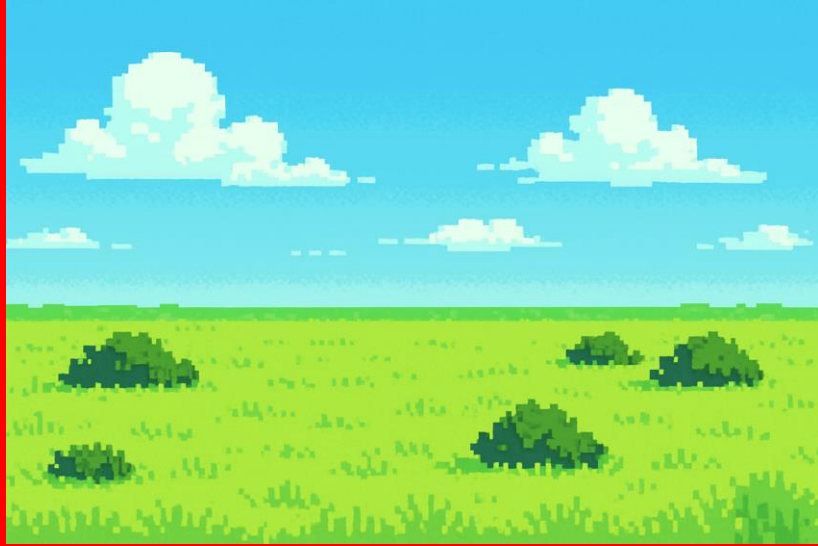
desativar o estado atual

```
void desativarEstadoAtual() {  
    esconderTodasImagens(imagens_globais);  
    esconderTodosOsTextos(textos_globais);  
}
```

Função de mudar de estado(apenas um pedaço dela)

```
void mudarEstado(EstadoJogo novo_estado) {  
    printf("Mudando estado de %d para %d\n", estado_atual, novo_estado);  
  
    desativarEstadoAtual();  
  
    estado_anterior = estado_atual;  
    estado_atual = novo_estado;  
  
    switch(novo_estado) {  
        case ESTADO_MENU_PRINCIPAL: carregarMenu(); break;  
        case ESTADO_TELA_CARREGAMENTO: carregarTelaCarregamento(); break;  
        case ESTADO_VILA: carregarVila(); break;  
        case ESTADO_MAPA: carregarMapa(); break;  
        case ESTADO_PANTANO: carregarPantano(); break;  
        case ESTADO_PLANICIE: carregarPlanicie(); break;  
        case ESTADO_TERREO: carregarTerreo(); break;  
        case ESTADO_DIRECOES_PANTANO: carregarDirecoesPantano(); break;  
        case ESTADO_DIRECOES_PLANICIE: carregarDirecoesPlanicie(); break;  
        case ESTADO_NORTE_PANTANO: carregarNortePantano(); break;  
        case ESTADO_SUL_PANTANO: carregarSulPantano(); break;  
        case ESTADO_LESTE_PANTANO: carregarLestePantano(); break;  
        case ESTADO_OESTE_PANTANO: carregarOestePantano(); break;  
    }
```

Criação de novas telas



Implementação de novos Estados

```
if(estado_atual == ESTADO_MAPA) {  
    if(cliqueBotao(btnPantano, mouseX, mouseY)) {  
        mudarEstado(ESTADO_PANTANO);  
    }  
    if(cliqueBotao(btnPlanicie, mouseX, mouseY)) {  
        mudarEstado(ESTADO_PLANICIE);  
    }  
    if(cliqueBotao(btnCastelo, mouseX, mouseY)) {  
        mudarEstado(ESTADO_TERREO);  
    }  
    if(cliqueBotao(btnVoltarMapa, mouseX, mouseY)) {  
        mudarEstado(ESTADO_VILA);  
    }  
}
```


Planície

```
if(estado_atual == ESTADO_PLANICIE) {  
    if(cliqueBotao(btnRegioesPlanicie, mouseX, mouseY)) {  
        mudarEstado(ESTADO_ROSA_DOS_VENTOS);  
    }  
    if(cliqueBotao(btnVoltarPlanicie, mouseX, mouseY)) {  
        mudarEstado(ESTADO_MAPA);  
    }  
}
```

Pântano

```
if(estado_atual == ESTADO_PANTANO) {  
    if(cliqueBotao(btnRegioesPantano, mouseX, mouseY)) {  
        mudarEstado(ESTADO_ROSA_DOS_VENTOS);  
    }  
  
    if(cliqueBotao(btnVoltarPantano, mouseX, mouseY)) {  
        mudarEstado(ESTADO_MAPA);  
    }  
}
```

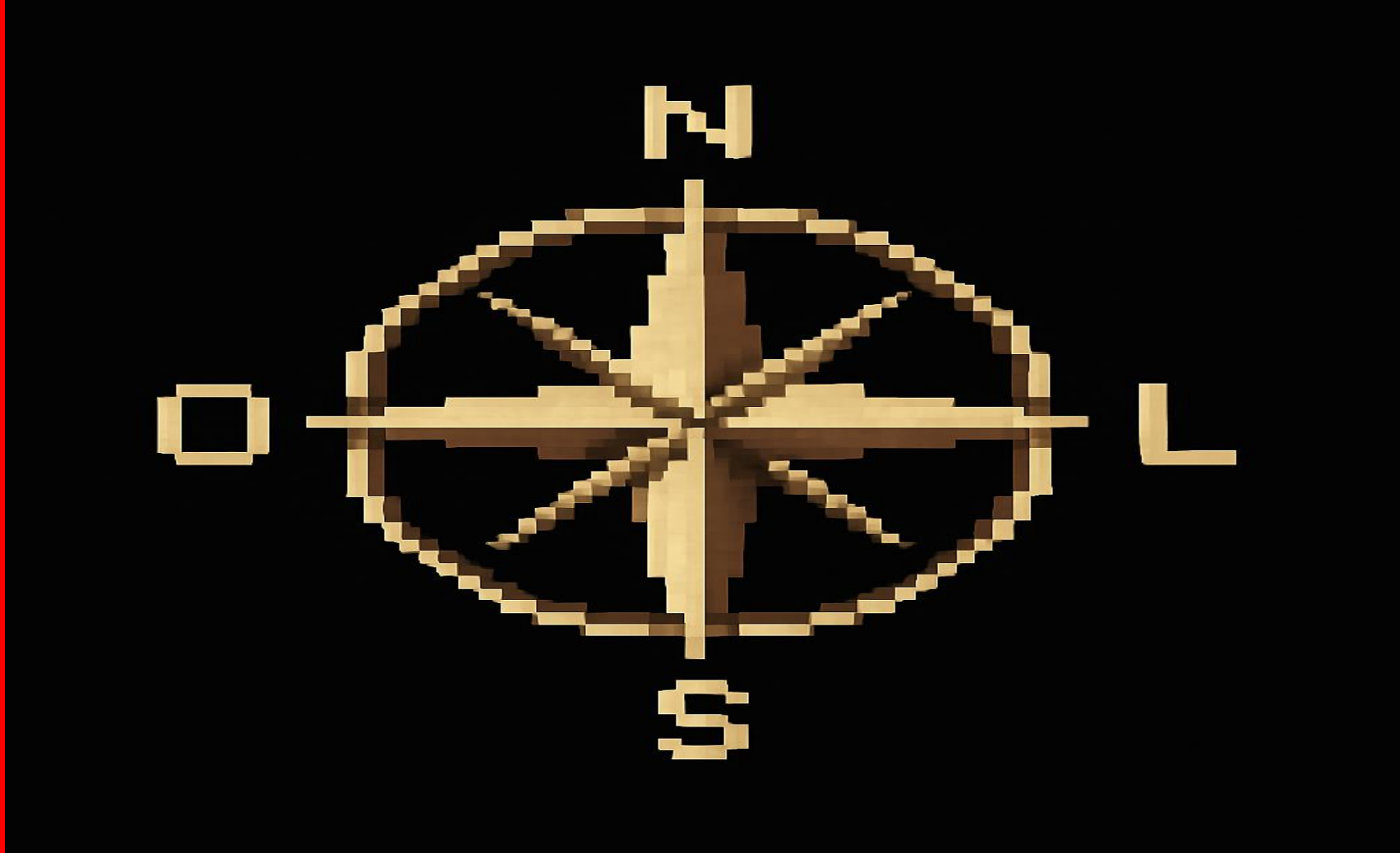
Estados do Castelo

```
if(estado_atual == ESTADO_TERREO) {  
    if(cliqueBotao(btnPrimeiroAndar, mouseX, mouseY)) {  
        mudarEstado(ESTADO_PRIMEIRO_ANDAR);  
    }  
    if(cliqueBotao(btnSegundoAndar, mouseX, mouseY)) {  
        mudarEstado(ESTADO_SEGUNDO_ANDAR);  
    }  
    if(cliqueBotao(btnSalaRei, mouseX, mouseY)) {  
        mudarEstado(ESTADO_SALA_REI);  
    }  
    if(cliqueBotao(btnVoltarCastelo, mouseX, mouseY)) {  
        mudarEstado(ESTADO_MAPA);  
    }  
}
```

Estado do primeiro andar(exemplo)

```
if(estado_atual == ESTADO_PRIMEIRO_ANDAR) {  
    if(cliqueBotao(btnTerreo, mouseX, mouseY)) {  
        mudarEstado(ESTADO_TERREO);  
    }  
    if(cliqueBotao(btnSegundoAndar, mouseX, mouseY)) {  
        mudarEstado(ESTADO_SEGUNDO_ANDAR);  
    }  
    if(cliqueBotao(btnSalaRei, mouseX, mouseY)) {  
        mudarEstado(ESTADO_SALA_REI);  
    }  
    if(cliqueBotao(btnVoltarCastelo, mouseX, mouseY)) {  
        mudarEstado(ESTADO_MAPA);  
    }  
}
```

implementação do sistema de bússola



implementação do sistema de bússola(ainda em desenvolvimento)

```
if(estado_atual == ESTADO_ROSA_DOS_VENTOS) {  
    if(cliqueBotao(btnOeste, mouseX, mouseY)) {  
        mudarEstado(ESTADO_OESTE_PANTANO);  
    }  
}
```

```
Botao btnOeste, btnLeste, btnNorte, btnSul;
```

```
btnOeste = criarBotao(0, 200, 200, 100, "", corTransparente, corTransparente, corTransparente);
```

